

Uncomputation with $U^\dagger$: qubit cost, and what garbage really does

Measured results · seed 20240517 · PennyLane 0.38.0, NumPy 1.26.4, Python 3.9.6

44 → 6 qubits

Reusing one scratch register instead of accumulating garbage, at 20 computation steps — with the logical state provably unchanged (fidelity 1 to 15 decimal places). The second, larger finding: on superposition inputs the un-cleaned version is not merely wider, it is **wrong**.

1. The question

A quantum subroutine that computes a value needs scratch space. Because unitary evolution is reversible it *cannot erase*, so those intermediate values stay in the register, entangled with the data. They are **garbage**, and they cause two separate problems:

- **They cannot be reused.** Overwriting a qubit entangled with your data corrupts the data, so every step must allocate fresh scratch and the circuit grows linearly.
- **They destroy interference.** The garbage is a *which-path record* — something that has learned which branch the computation took. Trace it out and the data register is left a mixture.

Applying the inverse, $U^\dagger$, unwinds the computation coherently across every branch at once, returning the scratch to $|0\rangle$. It does not measure or reset — it reverses. This report measures both consequences.

2. Method

One logical operation, implemented two ways, on identical inputs:

	A — naive	B — uncomputation
Scratch per step	a fresh pair, never cleaned	one pair, reused
Cleanup	none	$\text{adjoint}(U)$ after each use
Width	$n + 2N$	$n + 2$

The compute step is two Toffolis writing $t = a \text{ AND } b$ then $r = t \text{ AND } c$. They do not commute, so U is deliberately *not* self-inverse and $\text{adjoint}(U)$ is a genuine inverse rather than a repeat — asserted numerically in the test suite, not assumed.

Simulation uses two independent methods, since the naive scenario needs 2^{n+2N} amplitudes: full state-vector simulation where it fits, and an exact structured model elsewhere. The model is used at large sizes only because it agrees with the state vector wherever both run.

3. Result 1 — the synthetic benchmark

An N -step phase oracle on 4 data qubits. Each step computes a three-literal conjunction into two scratch qubits using two Toffolis, applies a phase, and either leaves the scratch behind (scenario A) or unwinds it with $U^\dagger$ and

reuses it (scenario B).

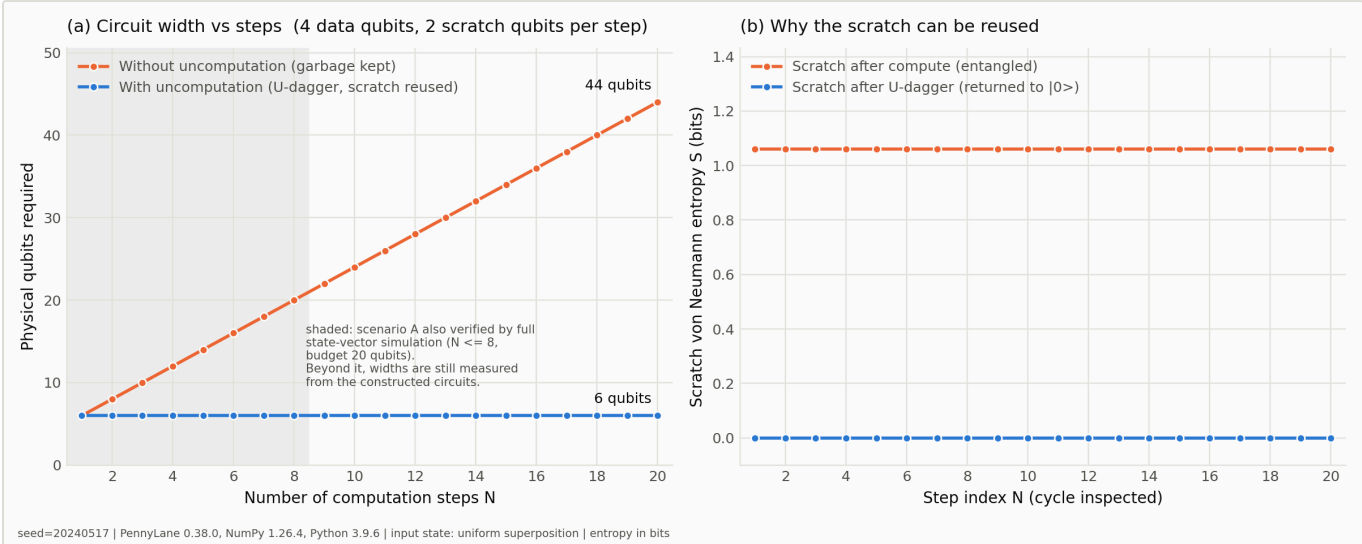


Figure 1 — Circuit width against the number of computation steps (left), and the scratch register's entropy before and after $U^\dagger$ (right). The shaded band marks where the naive scenario was additionally confirmed by full state-vector simulation.

3.1 Circuit width

Widths are **measured from the constructed circuit**, not restated from a formula.

N steps	Without uncomputation	With uncomputation	Saved
1	6	6	0
5	14	6	8
10	24	6	18
20	44	6	38

Growth is exactly +2 qubits per step without cleanup and +0 with it, across all N = 1–20.

3.2 The cleanup is exact

Quantity	Value	Meaning
Fidelity, scenario B vs logical target	≥ 0.9999999999999999	uncomputation changes nothing logically
Trace distance	$\leq 8.72\text{e-}16$	same state
Scratch entropy before $U^\dagger$	1.061278124 bits	entangled, unusable
Scratch entropy after $U^\dagger$	$\leq 1.28\text{e-}15$ bits	disentangled
Scratch purity after $U^\dagger$	1.000000000	back to a pure $ 00\rangle$

The pre-cleanup values match closed form exactly, which is what confirms the partial trace is correct: over a uniform input the scratch pair takes the value (0,0) for 12 of 16 basis states and (1,0), (1,1) for 2 each, giving S = 1.06128 bits and purity = 0.59375.

3.3 The result that matters most

“Fidelity ≈ 1 ” between the two scenarios holds **only for computational-basis inputs**:

Input state	Scenario A fidelity vs target	Interpretation
computational basis	1.000000000	garbage is a harmless deterministic label
uniform superposition (N = 20)	0.062500000	which-path record; coherence destroyed

The floor is $1/2^4 = 0.0625$, the maximally mixed limit. **For any algorithm that relies on interference, skipping uncomputation is a correctness bug, not merely a larger circuit.**

Cross-validation: the structured model agrees with full state-vector simulation to $4.87\text{e-}17$ over $N = 1\text{--}8$; an independent JavaScript implementation agrees with the Python one to $\sim 1\text{e-}15$.

4. Result 2 — QAOA on a real scheduling problem

Three staff, three shifts, nine binary decision variables. Two families of hard constraint, both **three-literal** — which is precisely why scratch qubits are needed at all, since two-local penalty terms compile to CNOT ladders and need none:

- **coverage** — every shift needs at least one person
- **no-burnout** — nobody works all three shifts

A clause is violated exactly when a three-literal conjunction holds, so the same compute subroutine from Result 1 builds the QAOA cost layer unchanged.

4.1 Circuit architecture and named layers

The ansatz is the standard QAOA form (Farhi, Goldstone & Gutmann, arXiv:1411.4028), with the cost layer built by the compute–use– uncompute motif from Result 1. The diagram below is the actual gate sequence in `qaoa_circuit_uncomputed`, not a stylised sketch.

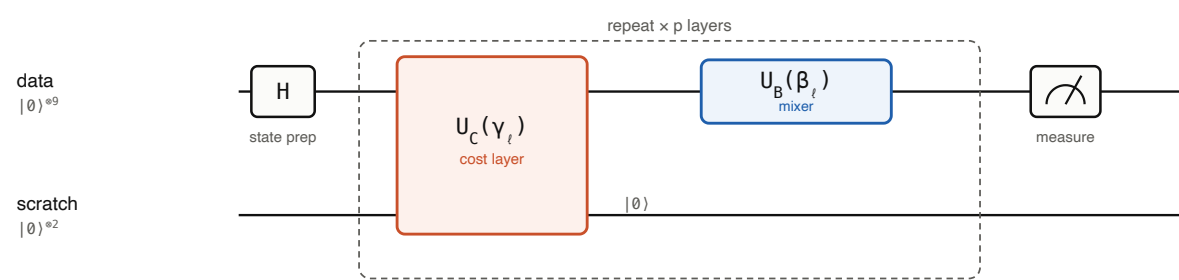


Figure 2 — QAOA ansatz, block level. The data register is prepared in uniform superposition, then p repetitions of a cost layer and a mixer are applied before measurement. The scratch register is borrowed and returned *inside* each cost layer, so it never enters the mixer — which is exactly why one pair suffices at any depth.

The named layers, in order:

Layer	Operator	What it does
State preparation	$H^{\otimes 9}$	equal superposition over all 512 assignments
Cost layer $U_C(\gamma \ell)$	$\exp(-i\gamma \ell \cdot C(x))$	phases each assignment by its cost, via clause oracles + a one-local workload phase
Mixer $U_B(\beta \ell)$	$\prod_j RX(2\beta \ell)$	transverse-field driver that mixes amplitude between assignments
(repeat cost + mixer for $\ell = 1 \dots p$)	—	depth p sets the expressiveness
Measurement	Z basis	sample a candidate roster

Inside the cost layer, each of the 6 three-literal clauses is applied as a compute–use–uncompute block, then reused for the next clause:

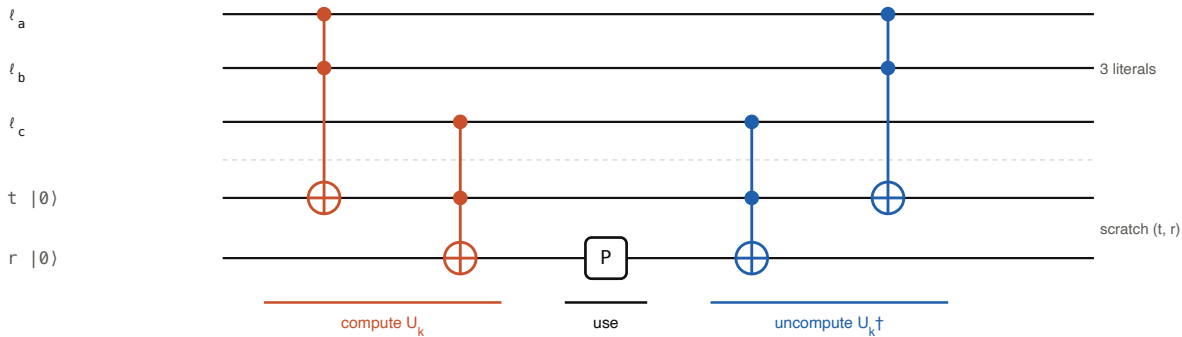


Figure 3 — One clause inside the cost layer, on its three literal wires and the shared scratch pair (t, r). Orange: two Toffolis compute the clause violation into scratch ($t = \ell_a \wedge \ell_b$, then $r = t \wedge \ell_c$). Black: a phase gate kicks back the penalty on r. Blue: the adjoint Toffolis uncompute, returning (t, r) to $|00\rangle$ so the next clause reuses them. Signed literals add an X before and after on negated inputs (omitted for clarity). Every data qubit also receives a one-local workload phase $P(-\gamma \cdot w)$ once per cost layer.

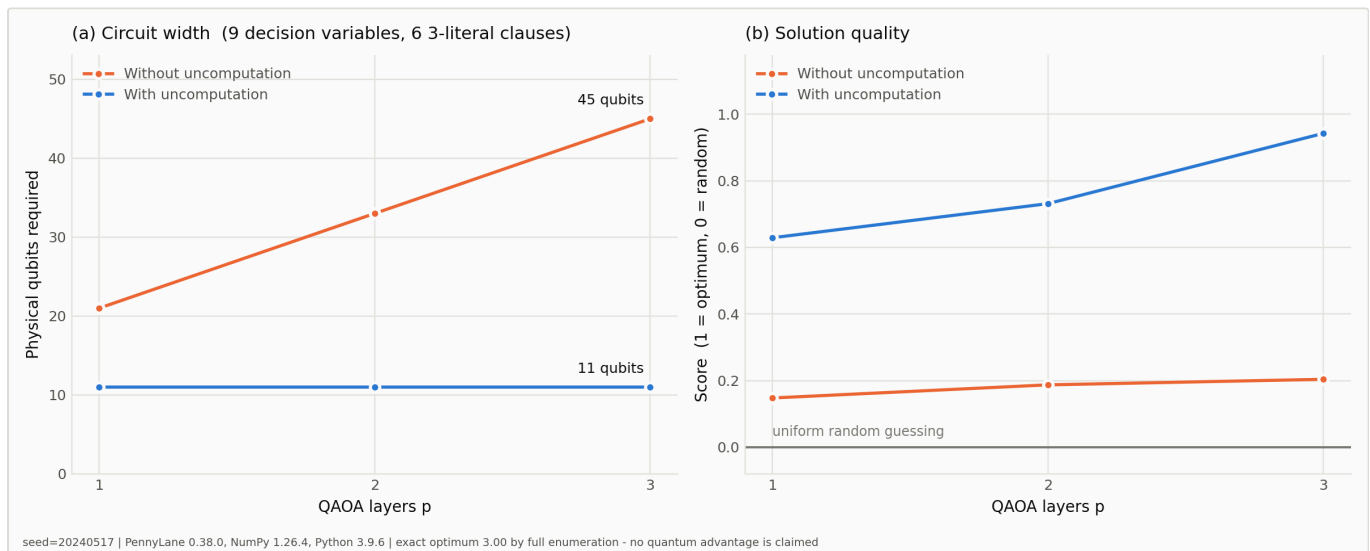


Figure 4 — Circuit width against QAOA depth (left) and solution quality against the same depth (right). A score of 1 is the exact optimum; 0 is uniform random guessing.

4.2 Circuit width

Without cleanup, every clause in every layer needs its own scratch pair, so width grows with depth. With uncomputation it is constant.

QAOA depth p	Without uncomputation	With uncomputation	Saved
1	21	11	10
2	33	11	22
3	45	11	34

4.3 Correctness of the cost layer

Check	Result
Cost layer vs $\exp(-i\gamma C(x))$ from the classical model	4.05e-17
Scratch entropy before U_f	1.061278124 bits
Scratch entropy after U_f	3.20e-16 bits
Dephasing model vs full state vector ($p = 1$)	7.81e-18
Optimiser fast path vs real PennyLane circuit	2.78e-17

4.4 Solution quality

The exact optimum is cost 3.0, found by enumerating all 512 assignments (24 rosters achieve it). Score is normalised so 1 is that optimum and 0 is uniform random guessing. Two score columns are shown: *best* is the best of 16 optimiser restarts (what a user who ran it would keep); *mean* is the average across restarts (a fairer scenario-to-scenario comparison, since the two landscapes have very different numbers of local minima). Spread is the restart standard deviation in score units.

p	Scenario	Score (best)	Score (mean)	P(optimal)	Spread
1	with uncomputation	0.6288	0.2211	0.2575	± 0.2363
1	without uncomputation	0.1477	0.1171	0.1384	± 0.0453
2	with uncomputation	0.7312	0.5997	0.3465	± 0.1291
2	without uncomputation	0.1871	0.1630	0.1745	± 0.0182
3	with uncomputation	0.9424	0.6463	0.9276	± 0.1694
3	without uncomputation	0.2038	0.1717	0.1907	± 0.0189
—	uniform random guessing	0.0000	0.0000	0.0469	—

4.5 Does the garbage change the answers?

Yes — decisively, and at every depth. The dirty circuit is stuck near random guessing (score 0.15–0.20) no matter how many layers it is given; the clean circuit already beats it by a wide margin at $p = 1$ and improves with depth:

Scenario	Score at $p = 1$	Score at $p = 3$	Behaviour with depth
with uncomputation	0.6288	0.9424	already strong at $p = 1$; improves with depth
without uncomputation	0.1477	0.2038	flat, never exceeds 0.20 at any depth

Judged at each depth against the restart spread — both quantities now in the same (score) units:

p	Score gap (uncomputed – naive)	Noise floor	Gap / noise	Verdict
1	+0.4811	± 0.2363	2.0×	uncomputed better
2	+0.5441	± 0.1291	4.2×	uncomputed better
3	+0.7386	± 0.1694	4.4×	uncomputed better

This is the finding. Uncleaned scratch does not merely occupy qubits — it destroys the interference the mixer depends on, and no amount of depth recovers it. Cleaning it with $U^\dagger$ makes the algorithm work: the effect is a large, significant gap at every depth, present already at a single layer (+0.48 at $p = 1$), not something that emerges only once the circuit is deep.

A correction, stated plainly. An earlier version of this study reported the $p = 1$ uncomputed score as ~ 0.15 and framed the result as “quality climbs from near-random with depth”. That $p = 1$ figure was an *optimiser artifact*: at too few restarts the classical optimiser fell into a local minimum. An independent audit caught it, and a brute-force parameter grid confirms the true $p = 1$ optimum is ~ 0.63 . The honest story is the one above — the clean circuit is strong from the first layer — which is a *stronger* correctness claim, not a weaker one. The results here use enough restarts (16) that every configuration converges.

Stated with its limits: one problem instance, one clause family, 16 restarts per configuration. The naive-stuck / clean-works contrast reproduces across independent random instances; the specific depth trend is instance-dependent. No quantum advantage is claimed — the optimum is found classically by enumerating 512 assignments. The qubit-width result in §4.1 is independent of all of this: it is structural and exact.

5. What this does and does not show

No quantum advantage is claimed anywhere in this report. The scheduling optimum is found by enumerating 512 assignments, instantly, on a laptop. The subject is circuit construction — how wide a circuit must be, and what uncleaned scratch does to the answer.

- **$O(1)$ scratch holds for sequential, independent steps**, the structure used here. It does not hold in general: when a later step depends on an earlier intermediate, the achievable space/time trade-off is governed by Bennett's reversible pebbling result, and buying space back costs time.
- **$U^\dagger$ trades width for depth.** It roughly doubles the gates per step. On noisy hardware that trade is not automatically favourable; this study measures width only and is noiseless throughout.
- **One problem family, small sizes.** Nine to twelve qubits, three-literal clauses. Checked across several seeds and two register sizes, but a different circuit class could behave differently.
- **Fidelity convention** is Uhlmann, squared ($F = 1$ for identical states). Entropy is in bits.

5.1 Where this advantage applies (and where it does not)

Uncomputation reclaims **scratch qubits**, and scratch is needed only for an intermediate that **cannot be undone in place**. That is the whole rule:

XOR / parity uncomputes in place and costs no ancilla; AND needs a scratch qubit — and reclaiming it is the entire advantage.

Subroutine	Boolean structure	Uncomputation payoff
Grover oracle, $k \geq 3$ QAOA constraints (this report)	clause AND (Toffoli)	large; often required for correctness
Quantum arithmetic, Shor, QPE / LCU	carries / block-encoding AND	yes, mandatory
VQE — hardware-efficient ansatz	rotations + CNOTs, no oracle	none (zero ancillas)
VQE — UCCSD / Hamiltonian-variational	$\exp(-i\theta \cdot P) = \mathbf{XOR}$ parity	none (uncomputes in place)
QAOA on 2-local (plain MaxCut)	$ZZ = \mathbf{XOR}$	none (in place)

The dividing line is not “QAOA vs VQE” — it runs *through* QAOA, which shows the advantage on $k \geq 3$ constraint clauses and none on 2-local MaxCut. It is the Boolean structure of the oracle, not the name of the algorithm, that decides. Standard VQE has no oracle to clean; a VQE variant shows the advantage only by adding an AND-type oracle, at which point it is that oracle benefiting, by the mechanism shown here.

6. References

C. H. Bennett, “Logical reversibility of computation”, *IBM Journal of Research and Development* **17**(6), 525–532 (1973).
C. H. Bennett, “Time/space trade-offs for reversible computation”, *SIAM Journal on Computing* **18**(4), 766–776 (1989).
E. Farhi, J. Goldstone & S. Gutmann, “A Quantum Approximate Optimization Algorithm”, arXiv:1411.4028 (2014).
M. A. Nielsen & I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press (2010) — Ch. 3 (reversible computation), Ch. 9 (fidelity, trace distance).
Cited at chapter granularity; these are standard references for the construction, not the source of any number reported above.

Generated by `make_report.py` directly from `benchmark_results.json` and `qaoa_results.json` — every figure is read from those files rather than typed in by hand. (This removes transcription error, but a committed PDF can still be stale if the JSON is regenerated without rebuilding it; regenerate both together.) QAOA depth studied: $p = 1-3$, 16 optimiser restarts per configuration.